TRƯỜNG ĐẠI HỌC CỬU LONG • KHOA CNTT – TRÍ TUỆ NHÂN TẠO

BÀI GIẢNG KIỂM THỬ PHẦN MỀM

CHƯƠNG 5

KIỂM THỬ TỰ ĐỘNG (AUTOMATION TESTING)

Khái niệm – Lợi ích – So sánh với kiểm thử thủ công – Quy trình – Công cụ Web/API/Mobile

Thời lượng: **4 tiết** (1 tiết = 45 phút)

Biên soạn: ThS. Lôi Thị Tú Trân

Vĩnh Long – 2026

Mục tiêu chương học

Kiến thức

Trình bày được khái niệm, lợi ích, ưu điểm và hạn chế của kiểm thử tự động.

Kiến thức

So sánh được kiểm thử thủ công và kiểm thử tự động trên nhiều tiêu chí.

Kiến thức

Nhận biết các công cụ phổ biến cho kiểm thử Web, API và Mobile.

Kỹ năng

Quyết định được nên hay không nên tự động hoá một trường hợp kiểm thử cụ thể.

Kỹ năng

Mô tả và triển khai được quy trình xây dựng kiểm thử tự động cho một chức năng.

Thái độ

Hiểu rằng tự động hoá là **công cụ hỗ trợ**, không thay thế hoàn toàn con người.

Chuẩn tham chiếu: **ISTQB CTFL v4.0** – Chapter 6: Test Tools; tài liệu chính thức của Selenium, Postman, JMeter, Appium.

Nội dung chương 5

- **5.1** Tổng quan về kiểm thử tự động (Automation Testing)
- **5.2** Lợi ích của kiểm thử tự động
- **5.3** So sánh kiểm thử tự động và kiểm thử thủ công
- **5.4** Khi nào NÊN áp dụng kiểm thử tự động
- **5.5** Khi nào KHÔNG nên áp dụng kiểm thử tự động
- **5.6** Quy trình xây dựng kiểm thử tự động (Automation Testing Life Cycle)
- **5.7** Công cụ kiểm thử tự động: Web – API – Mobile
- **5.8** Rủi ro thường gặp và các thực hành tốt (best practices)
- **5.9** Bài tập vận dụng + Bài tập tự luyện + Câu hỏi ôn tập

THỜI LƯỢNG: ≈ 0,75 TIẾT

5.1

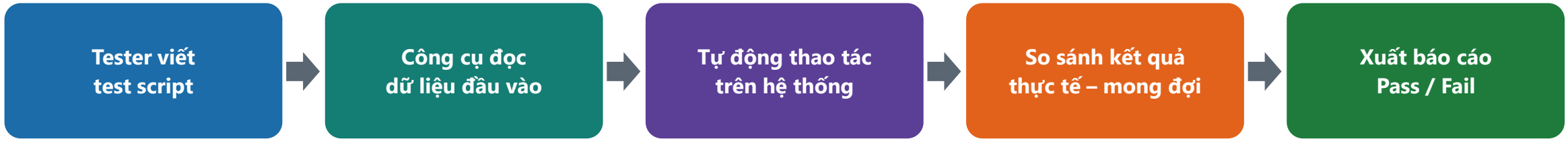
Tổng quan và lợi ích của kiểm thử tự động

- Khái niệm
- Cơ chế hoạt động
- Sáu nhóm lợi ích

Kiểm thử tự động là gì?

Kiểm thử tự động (Automation Testing) là quá trình sử dụng **công cụ và chương trình phần mềm** để thực hiện các trường hợp kiểm thử một cách tự động, thay vì tester thao tác thủ công trên hệ thống.

CƠ CHẾ HOẠT ĐỘNG



VÍ DỤ – kiểm thử chức năng Đăng nhập

Thủ công: tester tự nhập username, nhập password, nhấn nút, quan sát kết quả (≈ 40 giây/lần).

Tự động: viết 1 script; chạy 100 bộ dữ liệu trong ≈ 2 phút, lặp lại bao nhiêu lần cũng được.

ĐẶC ĐIỂM NỔI BẬT

- Tốc độ thực thi nhanh
- Khả năng lặp lại cao
- Độ chính xác ổn định
- Giảm phụ thuộc vào con người
- **Đòi hỏi tester có kiến thức lập trình** để xây dựng và duy trì script.

Sáu nhóm lợi ích của kiểm thử tự động

① Tiết kiệm thời gian

Thực hiện hàng loạt test case trong thời gian rất ngắn. Đặc biệt hiệu quả với **kiểm thử hồi quy** khi hệ thống cập nhật nhiều lần.

② Độ chính xác cao

Script luôn thực hiện **đúng như đã lập trình**, không nhập sai dữ liệu, không bỏ sót bước, không đánh giá sai kết quả vì mệt mỏi.

③ Khả năng tái sử dụng

Một script viết một lần có thể **dùng lại cho nhiều phiên bản** của phần mềm, tiết kiệm công sức đáng kể qua thời gian.

④ Hỗ trợ CI/CD

Mỗi lần lập trình viên đẩy mã nguồn (build), hệ thống **tự động chạy test** ⇒ phát hiện lỗi sớm, giảm rủi ro khi triển khai.

⑤ Tăng độ bao phủ

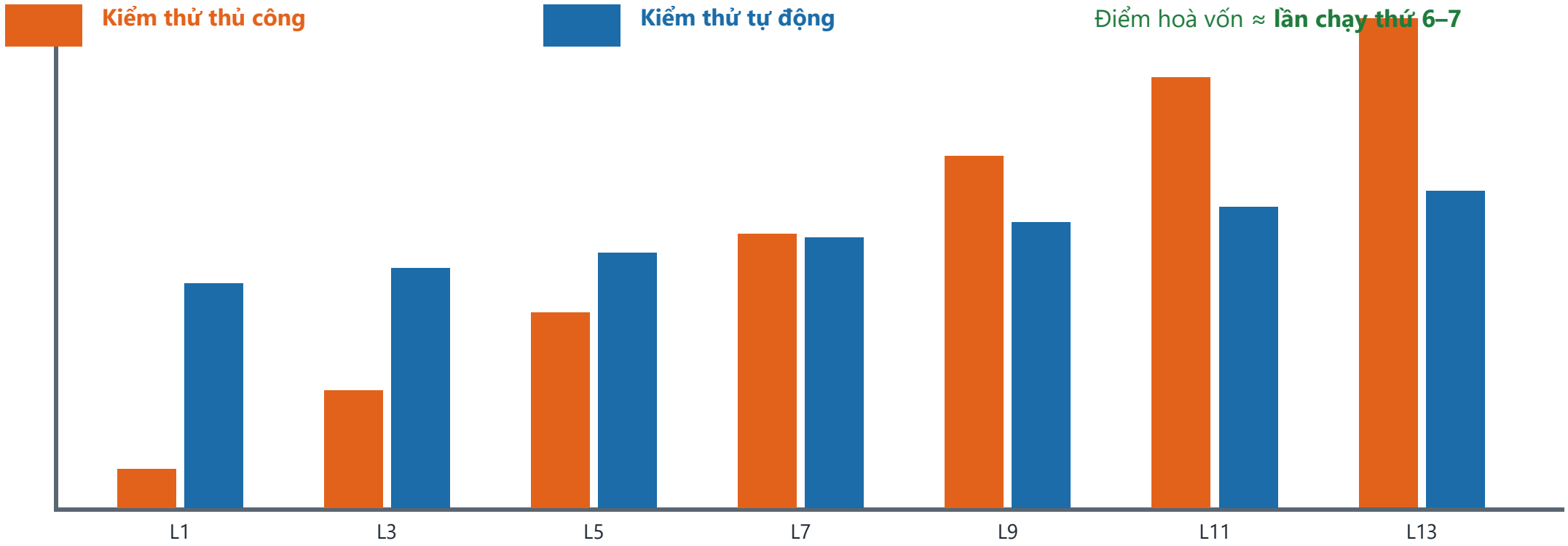
Dễ dàng kiểm thử **hàng trăm bộ dữ liệu** đầu vào (data-driven testing) – điều mà kiểm thử thủ công rất khó thực hiện đầy đủ.

⑥ Tiết kiệm chi phí dài hạn

Chi phí đầu tư ban đầu cao nhưng khi số test case tăng lên, chi phí trên mỗi lần chạy giảm mạnh ⇒ **ROI dương** sau một số vòng chạy nhất định.

Điểm hoà vốn của tự động hoá (Automation ROI)

Chi phí tích luỹ theo số lần thực thi bộ kiểm thử



Kết luận: tự động hoá chỉ có lợi khi bộ test được **chạy lại nhiều lần**. Với chức năng chỉ kiểm thử 1–2 lần, kiểm thử thủ công **rẻ hơn và nhanh hơn**.

THỜI LƯỢNG: ≈ 0,75 TIẾT

5.3

So sánh kiểm thử thủ công và kiểm thử tự động

- Đặc điểm mỗi phương pháp
- Bảng so sánh
- Ưu điểm và hạn chế

Đặc điểm của hai phương pháp

MANUAL TESTING – Kiểm thử thủ công

- Tester **đóng vai người dùng cuối**, trực tiếp thực hiện các test case đã định nghĩa trước
- **Không dùng công cụ tự động** – mọi bước thực hiện bằng tay
- Đòi hỏi **kỹ năng phân tích**: hiểu rõ yêu cầu, suy luận và tìm lỗi tiềm ẩn
- Tester tự thực thi test và tự lập báo cáo kết quả

AUTOMATION TESTING – Kiểm thử tự động

- Sử dụng **công cụ hoặc framework** để chạy test case tự động
- **Dựa trên script**: phải viết mã mô phỏng thao tác và kiểm chứng kết quả
- Có thể **lấy dữ liệu từ file bên ngoài** (Excel, CSV, JSON), so sánh với kết quả mong đợi và **tự xuất báo cáo**
- Phù hợp nhất với **kiểm thử hồi quy** và các kịch bản lặp lại nhiều lần

Bảng so sánh kiểm thử thủ công và kiểm thử tự động

Tiêu chí	Kiểm thử thủ công	Kiểm thử tự động
Cách thực hiện	Tester thao tác trực tiếp	Máy chạy script đã lập trình sẵn
Tốc độ	Chậm	**Nhanh** – gấp hàng chục lần
Độ chính xác	Dễ sai sót do yếu tố con người	**Chính xác cao**, nhất quán qua nhiều lần chạy
Chi phí ban đầu	Thấp	Cao (công cụ, đào tạo, xây dựng framework)
Chi phí dài hạn	Cao (tốn nhân lực mỗi lần chạy)	Thấp (chạy lại gần như miễn phí)
Khả năng lặp lại	Khó, dễ mệt mỏi và bỏ sót	**Dễ**, chạy bao nhiêu lần cũng được
Kỹ năng yêu cầu	Không bắt buộc biết lập trình	**Bắt buộc** biết lập trình
Ghi lại quá trình	Khó ghi lại đầy đủ	Tự động lưu log, ảnh chụp, video
Phù hợp với	UI/UX, kiểm thử thăm dò, test linh hoạt	Hồi quy, test lặp lại, hiệu năng, API

Ưu điểm và hạn chế của kiểm thử thủ công

ƯU ĐIỂM

- Tester có **phản hồi trực quan** nhanh và chính xác về trải nghiệm người dùng
- **Ít tốn kém ban đầu** – không cần chi ngân sách cho công cụ và hạ tầng tự động hoá
- Có thêm **khả năng phán đoán của con người** – phát hiện được điều bất thường ngoài kịch bản
- Yêu cầu thay đổi **không làm kiểm thử trở nên quá phức tạp**

HẠN CHẾ

- **Ít tin cậy hơn** vì do con người thực hiện ⇒ dễ xảy ra sai sót, bỏ sót bước
- Quá trình kiểm thử **không được ghi lại** đầy đủ, khó tái hiện và khó kiểm chứng
- Một số tác vụ **rất khó hoặc không thể** thực hiện thủ công: kiểm thử hiệu năng, kiểm thử tải với hàng nghìn người dùng đồng thời
- Tốn nhiều thời gian khi phải lặp lại nhiều lần

Ưu điểm và hạn chế của kiểm thử tự động

ƯU ĐIỂM

- Quá trình kiểm thử diễn ra **nhanh chóng và hiệu quả** hơn
- Có thể tìm ra **nhiều lỗi hơn** so với thao tác thủ công trong cùng thời gian
- Quá trình test **được ghi lại**, cho phép **tái sử dụng** hàng loạt hoạt động thử nghiệm
- Công cụ hoạt động **không mệt mỏi**, không bỏ sót ngay cả đơn vị nhỏ nhất
- Hỗ trợ **nhiều loại ứng dụng** và mở rộng phạm vi kiểm tra

HẠN CHẾ

- Khó đánh giá **khía cạnh trực quan** của giao diện: màu sắc, phông chữ, độ tương phản, kích thước nút
- Công cụ có thể **đắt tiền**, làm tăng chi phí dự án
- Công cụ **chưa hoàn hảo** – vẫn có giới hạn kỹ thuật (upload file, iframe, captcha)
- **Bảo trì tốn kém** – giao diện thay đổi làm script hỏng hàng loạt

Kết luận: không phương pháp nào thay thế hoàn toàn phương pháp còn lại. **Kết hợp cả hai** mới tối ưu được hiệu quả, chất lượng và chi phí.

THỜI LƯỢNG: ≈ 0,5 TIẾT

5.4

Khi nào nên và không nên áp dụng kiểm thử tự động

- Sáu trường hợp nên áp dụng
- Sáu trường hợp không nên áp dụng

NÊN áp dụng kiểm thử tự động khi...

Chức năng lặp lại nhiều lần

Đăng nhập, đăng ký, tìm kiếm, thanh toán – được kiểm tra trong nhiều lần phát triển ⇒ tiết kiệm lớn.

Kiểm thử hồi quy

Sau mỗi lần nâng cấp phải chạy lại toàn bộ test cũ – thủ công khó bảo đảm về mặt thời gian.

Dự án quy mô lớn

Nhiều chức năng, nhiều module, nhiều phiên bản ⇒ thủ công tốn thời gian và dễ sai sót.

Kiểm thử hiệu năng

Load test, stress test – **bắt buộc** phải tự động vì cần mô phỏng hàng nghìn người dùng đồng thời.

Kiểm thử API và dữ liệu

Gửi nhiều request với nhiều bộ dữ liệu khác nhau để kiểm tra tính đúng đắn và độ ổn định.

Hệ thống ổn định, ít thay đổi

Script xây dựng một lần dùng được lâu dài, không phải sửa nhiều ⇒ chi phí bảo trì thấp.

KHÔNG nên áp dụng kiểm thử tự động khi...

Chỉ kiểm tra 1–2 lần

Công sức xây dựng script lớn hơn lợi ích thu được; thao tác thủ công nhanh và linh hoạt hơn.

Giao diện thay đổi liên tục

Đặc biệt ở giai đoạn đầu phát triển – thay đổi nhỏ về UI cũng làm script hỏng, phải sửa liên tục.

Dự án nhỏ, thời gian ngắn

Xây dựng hệ thống tự động hoá không mang lại nhiều giá trị, thậm chí làm chậm tiến độ.

Yêu cầu chưa ổn định

Kịch bản phải cập nhật liên tục theo thay đổi ⇒ chi phí bảo trì cao và khó kiểm soát.

Kiểm thử trải nghiệm người dùng

Đánh giá giao diện, tính thân thiện, cảm nhận khi sử dụng – **mang tính chủ quan**, cần con người.

Thiếu nguồn lực hoặc kỹ năng

Đội ngũ chưa có kinh nghiệm lập trình hoặc chưa làm chủ công cụ ⇒ gây khó khăn nhiều hơn lợi ích.

Nguyên tắc chung: tự động hoá chỉ phát huy hiệu quả khi được áp dụng **đúng hoàn cảnh**. Quyết định phải dựa trên đặc điểm cụ thể của dự án, không áp dụng cứng nhắc.

Quy trình xây dựng kiểm thử tự động (Automation Testing Life Cycle)

BƯỚC 1

① Lựa chọn công cụ kiểm thử

Phù hợp loại ứng dụng (web/API/mobile), ngôn ngữ lập trình và năng lực của nhóm.

BƯỚC 2

② Thiết kế kịch bản kiểm thử

Mô tả thao tác người dùng dưới dạng chương trình: dữ liệu đầu vào, các bước, kết quả mong đợi.

BƯỚC 3

③ Thực thi kiểm thử tự động

Chạy script trên hệ thống, có thể chạy nhiều lần và trên nhiều môi trường khác nhau.

BƯỚC 4

④ Phân tích kết quả kiểm thử

Đánh giá chất lượng phần mềm, ghi nhận & báo cáo lỗi, cải tiến kịch bản kiểm thử.

Trong thực tế còn có bước ⓪ **Đánh giá tính khả thi** (chọn test case nào đáng tự động hoá) và bước ⑤ **Bảo trì script** – chiếm phần lớn công sức về lâu dài.

Chi tiết từng bước trong quy trình

① LỰA CHỌN CÔNG CỤ KIỂM THỬ

- Căn cứ: loại ứng dụng, ngôn ngữ lập trình nhóm thành thạo, chi phí bản quyền, khả năng tích hợp CI/CD, cộng đồng hỗ trợ
- Chọn sai công cụ ⇒ khó triển khai và làm **giảm hiệu quả kiểm thử**

② THIẾT KẾ KỊCH BẢN KIỂM THỬ (TEST SCRIPT)

- Xác định rõ **dữ liệu đầu vào – các bước thực hiện – kết quả mong đợi**
- Script nên **rõ ràng, dễ hiểu và có khả năng tái sử dụng** để thuận tiện cho việc bảo trì

③ THỰC THI KIỂM THỬ TỰ ĐỘNG

- Chạy trên nhiều môi trường; kết quả (Pass/Fail) được **ghi nhận tự động** kèm log và ảnh chụp màn hình

④ PHÂN TÍCH KẾT QUẢ KIỂM THỬ

- Phân biệt **lỗi thật của sản phẩm** với **lỗi của script** (false failure); ghi nhận và báo cáo cho nhóm phát triển
- Đánh giá hiệu quả kịch bản và đề xuất cải tiến

THỜI LƯỢNG: ≈ 0,75 TIẾT

5.7

Công cụ kiểm thử tự động

- Công cụ kiểm thử Web
- Công cụ kiểm thử API
- Công cụ kiểm thử Mobile

Công cụ kiểm thử giao diện Web

Selenium

Bộ công cụ mã nguồn mở miễn phí cho ứng dụng web.

Tự động hoá thao tác trên trình duyệt: nhập dữ liệu, nhấn nút, kiểm tra kết quả.

Hỗ trợ **C#, Java, Python, Ruby, JavaScript**.

Dùng nhiều cho kiểm thử chức năng web và **kiểm thử hồi quy**.

⇒ Học chi tiết ở **Chương 6**.

Cypress

Công cụ **hiện đại** dành cho ứng dụng web phía frontend dùng JavaScript Framework.

Tốc độ chạy nhanh, dễ cài đặt.

Có khả năng **hiển thị trực tiếp quá trình kiểm thử** (time-travel debugging), giúp dễ quan sát và phát hiện lỗi.

Hạn chế: chủ yếu hỗ trợ JavaScript/TypeScript.

Playwright

Framework mã nguồn mở **do Microsoft phát triển**, tự động hoá **end-to-end (E2E)** trên Chromium, Firefox và WebKit.

- Tự động chờ (**auto-waiting**) ⇒ giảm test "flaky"
- Chạy **song song** nhiều test case
- Quay video, chụp ảnh màn hình
- Đa ngôn ngữ: JS/TS, Python, Java, C#; hỗ trợ cả kiểm thử API.

Công cụ kiểm thử API

POSTMAN

- Công cụ **phổ biến nhất hiện nay** để phát triển, kiểm thử, chia sẻ và tài liệu hoá API
- Hoạt động như một **giao diện đồ hoạ (GUI)** giúp gửi yêu cầu HTTP (**GET, POST, PUT, DELETE**) tới server và nhận phản hồi **mà không cần viết code**
- Hỗ trợ: collection, biến môi trường, kiểm chứng tự động bằng script, chạy hàng loạt bằng **Newman** trong CI/CD
- Phù hợp: kiểm thử chức năng API, kiểm thử tích hợp hệ thống

JMETER

- Công cụ **kiểm thử hiệu năng** – đánh giá khả năng chịu tải của hệ thống
- Mô phỏng **nhiều người dùng truy cập đồng thời** để kiểm tra tốc độ phản hồi, độ ổn định và khả năng xử lý
- Hỗ trợ nhiều giao thức: **HTTP, FTP, JDBC** và các giao thức khác
- Cung cấp **biểu đồ và đồ thị** để phân tích kết quả kiểm thử
- Phù hợp: load testing, stress testing cho hệ thống nhiều người dùng

Kiểm thử API là quá trình kiểm tra và xác minh tính chính xác, hiệu năng và độ tin cậy của các giao diện lập trình ứng dụng – nơi các phần mềm và hệ thống khác nhau trao đổi dữ liệu với nhau.

Công cụ kiểm thử ứng dụng di động

APPIUM

- Công cụ kiểm thử tự động **phổ biến nhất cho ứng dụng di động**
- Hỗ trợ **cả hai nền tảng Android và iOS**
- Cho phép viết kịch bản bằng **Java, C#, Python...** mà **không cần thay đổi mã nguồn** của ứng dụng
- Kiểm thử được cả native app, hybrid app và mobile web
- Nhờ khả năng **đa nền tảng**, Appium được dùng rộng rãi trong các dự án mobile hiện đại

KATALON STUDIO

- Công cụ có **giao diện trực quan**, hỗ trợ kiểm thử **Web, API và ứng dụng di động** trong cùng một nền tảng
- Xây dựng **trên nền Selenium và Appium** nhưng được **đơn giản hoá** để dễ tiếp cận
- Cho phép xây dựng kịch bản bằng **kéo–thả** hoặc dùng **từ khoá có sẵn** ⇒ không yêu cầu nhiều kiến thức lập trình
- Hỗ trợ **quản lý test case, thực thi và tạo báo cáo** thuận tiện
- Thường dùng trong **giảng dạy** và các dự án cần triển khai kiểm thử nhanh

Bảng tổng hợp lựa chọn công cụ theo loại ứng dụng

Loại ứng dụng / mục tiêu	Công cụ đề xuất	Lý do chính
Web – kiểm thử chức năng & hồi quy đa trình duyệt	**Selenium** (C#/Java/Python)	Mã nguồn mở, đa trình duyệt, đa ngôn ngữ, cộng đồng lớn, dễ tích hợp CI/CD
Web frontend JavaScript, cần phản hồi nhanh	**Cypress**	Cài đặt nhanh, debug trực quan, phù hợp nhóm frontend
Web E2E hiện đại, cần chạy song song	**Playwright**	Auto-waiting, đa trình duyệt kể cả WebKit, quay video, hỗ trợ cả API
API – kiểm thử chức năng	**Postman** (+ Newman)	GUI dễ dùng, không cần code, chạy được trong pipeline CI
Hiệu năng – load/stress test	**JMeter**	Mô phỏng hàng nghìn người dùng, nhiều giao thức, báo cáo chi tiết
Mobile – Android & iOS	**Appium**	Đa nền tảng, không cần sửa mã nguồn ứng dụng
Nhóm ít kinh nghiệm lập trình, cần triển khai nhanh	**Katalon Studio**	Giao diện kéo–thả, từ khoá có sẵn, hỗ trợ cả Web/API/Mobile

Rủi ro thường gặp và thực hành tốt trong kiểm thử tự động

Rủi ro: Test "flaky"

Test lúc pass lúc fail do chờ đợi không ổn định.

Khắc phục: dùng **explicit wait** thay cho `Thread.Sleep`; tránh phụ thuộc thời gian cố định.

Rủi ro: Locator dễ vỡ

Dùng XPath dài, phụ thuộc vị trí ⇒ đổi giao diện là hỏng.

Khắc phục: ưu tiên **ID**, thêm thuộc tính `data-testid` phối hợp với developer.

Rủi ro: Chi phí bảo trì cao

Script trùng lặp, khó sửa.

Khắc phục: áp dụng mẫu **Page Object Model (POM)**, tách dữ liệu ra khỏi script.

Rủi ro: Tự động hoá sai chỗ

Tự động hoá test chỉ chạy 1 lần, hoặc test UI cho logic nghiệp vụ.

Khắc phục: tuân thủ **kim tự tháp kiểm thử** – ưu tiên unit và API.

Rủi ro: Test không độc lập

Test A phải chạy trước test B ⇒ không chạy song song được.

Khắc phục: mỗi test **tự chuẩn bị và dọn dẹp dữ liệu** của mình.

Rủi ro: Không ai đọc báo cáo

Bộ test đỏ liên tục, cả nhóm bỏ qua.

Khắc phục: tích hợp CI, **chặn merge khi test fail**, phân công người trực xử lý.

THỜI LƯỢNG: ≈ 1,25 TIẾT

5.9

Bài tập vận dụng và câu hỏi ôn tập

- 5 bài tập
- 10 bài tập tự luyện
- 6 câu hỏi ôn tập

Lựa chọn phương pháp kiểm thử

- Một công ty phần mềm đang phát triển các hệ thống sau:
  - **A.** Website bán hàng trực tuyến.
  - **B.** Ứng dụng ngân hàng trên điện thoại.
  - **C.** Hệ thống quản lý thư viện nội bộ.
  - **D.** Website giới thiệu doanh nghiệp, chỉ cập nhật một lần.
- **Yêu cầu:**
  - a) Xác định với từng hệ thống nên ưu tiên kiểm thử thủ công, kiểm thử tự động hay kết hợp cả hai.
  - b) Giải thích lý do lựa chọn và đề xuất loại kiểm thử phù hợp.tr9

Xây dựng quy trình kiểm thử tự động

- Nhóm của bạn được giao xây dựng hệ thống kiểm thử tự động cho chức năng **Đăng nhập** của một website thương mại điện tử.
- **Yêu cầu:**
  - a) Mô tả quy trình xây dựng kiểm thử tự động từ khi lựa chọn công cụ đến khi phân tích kết quả.tr16
  - b) Xác định dữ liệu kiểm thử cần chuẩn bị.no
  - c) Xây dựng **ít nhất 05 kịch bản kiểm thử (Test Script)** dưới dạng mô tả các bước thực hiện và kết quả mong đợi.no

Lựa chọn công cụ kiểm thử

- Một doanh nghiệp có ba dự án:
  - **A.** Website thương mại điện tử.
  - **B.** Hệ thống API phục vụ ứng dụng di động.
  - **C.** Ứng dụng học trực tuyến trên Android và iOS.
- **Yêu cầu:** Với mỗi dự án, hãy:
  - a) Lựa chọn công cụ phù hợp (Selenium, Cypress, Playwright 19, Postman, Jmeter 20, Appium, Katalon Studio 21).
  - b) Giải thích lý do lựa chọn.no
  - c) So sánh ưu điểm của công cụ được chọn với **ít nhất một công cụ khác**.no

Phân tích tình huống áp dụng Automation Testing

- Một dự án phát triển **website đặt vé máy bay** có các đặc điểm sau:
  - Có hơn **500 test case**.
  - Mỗi tuần phát hành **một phiên bản mới**.
  - Chức năng **đăng nhập, tìm kiếm chuyến bay và thanh toán** thường xuyên được kiểm thử.
  - **Giao diện quản trị** vẫn đang thay đổi liên tục.
- **Yêu cầu:**
  - a) Xác định chức năng nào nên tự động hoá tr7, chức năng nào nên kiểm thử thủ công và giải thích.
  - b) Đề xuất kế hoạch kết hợp Manual và Automation để tối ưu thời gian và chi phí.no

Đánh giá hiệu quả của kiểm thử tự động

- Một nhóm kiểm thử đã tự động hoá **250 test case hồi quy**.
  - Trước khi áp dụng kiểm thử tự động: mỗi lần chạy mất khoảng **18 giờ**.
  - Sau khi áp dụng kiểm thử tự động: toàn bộ test case được thực hiện trong **2 giờ**.
- **Yêu cầu:**
  - a) Tính thời gian tiết kiệm được sau mỗi lần thực hiện kiểm thử.
  - b) Phân tích các lợi ích mà kiểm thử tự động mang lại **ngoài** việc tiết kiệm thời gian.
  - c) Đề xuất các tiêu chí để đánh giá hiệu quả của hệ thống kiểm thử tự động sau khi triển khai.

Bộ 10 bài tập tự luyện (Chương 5)

- **Bài 1.** Nêu 5 tiêu chí lựa chọn một công cụ kiểm thử tự động cho dự án. Xếp theo thứ tự ưu tiên và giải thích.tr20-tr21
- **Bài 2.** Một test case chạy tay mất 6 phút; viết script mất 3 giờ; bảo trì 10 phút/lần chạy. Sau bao nhiêu lần chạy thì tự động hoá có lợi?1-2 lần
- **Bài 3.** Giải thích khái niệm test "flaky". Nêu 4 nguyên nhân và cách khắc phục từng nguyên nhân.tr23
- **Bài 4.** Vì sao nên tách dữ liệu kiểm thử ra khỏi test script? Trình bày khái niệm **data-driven testing**.tr6
- **Bài 5.** So sánh Selenium, Cypress và Playwright theo 5 tiêu chí. Nêu tình huống phù hợp nhất cho mỗi công cụ.tr19
- **Bài 6.** Trình bày mẫu thiết kế **Page Object Model (POM)**. Vì sao POM giúp giảm chi phí bảo trì?tr23
- **Bài 7.** Một bộ test tự động chạy 90 phút, trong đó 80 phút là test UI. Đề xuất cách rút ngắn còn dưới 30 phút.no
- **Bài 8.** Phân biệt kiểm thử API bằng Postman và kiểm thử hiệu năng bằng JMeter: mục tiêu, dữ liệu, tiêu chí đánh giá.tr20
- **Bài 9.** Nêu 5 loại kiểm thử **không nên** tự động hoá và giải thích lý do cho từng loại.no
- **Bài 10.** Thiết kế kế hoạch tích hợp bộ kiểm thử tự động vào pipeline CI/CD: chạy gì, khi nào, điều kiện chặn merge.no

Câu hỏi ôn tập chương 5

1. Trình bày khái niệm **kiểm thử tự động**. Phân tích những ưu điểm nổi bật so với kiểm thử thủ công.tr12

2. So sánh **Manual Testing** và **Automation Testing** về cách thực hiện, tốc độ, độ chính xác, chi phí, khả năng tái sử dụng và phạm vi áp dụng. Có nên thay thế hoàn toàn kiểm thử thủ công không? Giải thích.tr10

3. Trình bày các trường hợp **nên** tr14 và **không nên** tr15 áp dụng kiểm thử tự động. Cho ví dụ minh hoạ cho từng trường hợp.

4. Mô tả **quy trình xây dựng kiểm thử tự động**. tr18 Theo bạn bước nào ảnh hưởng lớn nhất đến chất lượng hệ thống kiểm thử tự động? Giải thích.

5. Giới thiệu các công cụ kiểm thử tự động phổ biến cho **Web, API và Mobile**. Nêu đặc điểm, ưu điểm và phạm vi sử dụng của từng công cụ.tr22

6. Giải thích khái niệm **điểm hoà vốn (ROI)**tr7 trong kiểm thử tự động. Vì sao không phải lúc nào tự động hoá cũng tiết kiệm chi phí?

Tóm tắt chương 5

- Kiểm thử tự động dùng **công cụ và script** để thực thi test case, mang lại tốc độ, độ chính xác và khả năng lặp lại cao.
- Tự động hoá **không miễn phí**: chi phí ban đầu và chi phí **bảo trì** rất đáng kể – chỉ có lợi khi bộ test được **chạy lại nhiều lần**.
- **Nên tự động hoá:** hồi quy, chức năng lặp lại, kiểm thử hiệu năng, API, hệ thống ổn định. **Không nên:** UI/UX, thăm dò, giao diện đang thay đổi, dự án chạy 1–2 lần.
- Quy trình gồm 4 bước: **chọn công cụ → thiết kế script → thực thi → phân tích kết quả**, cộng thêm bảo trì liên tục.
- Công cụ tiêu biểu: **Selenium/Cypress/Playwright** (Web), **Postman/JMeter** (API & hiệu năng), **Appium/Katalon** (Mobile).
- **Kết hợp thủ công và tự động** mới tối ưu: máy làm việc lặp lại, con người làm việc cần phán đoán và sáng tạo.

Đánh giá kết quả học tập – Kiểm thử tự động

BÀI QUIZ • 30%

- **15 câu trắc nghiệm**, thời gian 15 phút
- Làm vào **đầu buổi chiều** ngày 1, 2, 3
- Mỗi câu **0,67 điểm** – làm tròn thang 10
- Chấm chéo tại lớp, công bố đáp án ngay
- Lấy **trung bình 3 bài quiz**

BÀI KIỂM TRA VIẾT 45 PHÚT • 70%

- **20 câu trắc nghiệm + 1 bài tập vận dụng**
- Làm vào **đầu ngày 4**, phủ cả 6 chương
- Trắc nghiệm **5 điểm** – bài tập **5 điểm**
- Mỗi chương góp khoảng **3–4 câu** trắc nghiệm
- Bài tập vận dụng chọn **1 trong 6 đề** của các chương

CÁCH TÍNH ĐIỂM QUÁ TRÌNH

Điểm quá trình = **0,7 × điểm bài kiểm tra viết 45 phút** + **0,3 × trung bình 3 bài quiz**

Ví dụ: bài viết 8,0 và ba quiz lần lượt 7,0 – 8,0 – 9,0 (trung bình 8,0) → điểm quá trình = 0,7 × 8,0 + 0,3 × 8,0 = **8,0**

Quiz 15 câu – Chương 5 (phần 1/3)

Câu 1. Kiểm thử tự động KHÔNG thay con người làm việc nào?
A. Gõ phím và bấm chuột B. So sánh kết quả **C**. Thiết kế test case và chọn dữ liệu D. Chạy lặp lại nhiều lần

Câu 2. Một script tự động KHÔNG có lệnh kiểm chứng (assertion) thì:
A. Luôn Fail B. Luôn Pass, không chứng minh được gì C. Không chạy được D. Chạy nhanh hơn

Câu 3. "Điểm hoà vốn" của tự động hoá là:
A. Số test case tối thiểu B. Số lần chạy mà chi phí tự động bằng chi phí thủ công C. Tỷ lệ Pass 100% D. Số tester cần thiết

Câu 4. Chạy tay 10 phút/lần; viết script 120 phút; mỗi lần chạy script tốn 3 phút. Điểm hoà vốn khoảng:
A. 12 lần B. 18 lần C. 24 lần D. 40 lần

Câu 5. Trường hợp nào NÊN tự động hoá nhất?
A. Test chạy một lần duy nhất B. Test hồi quy chạy sau mỗi bản build C. Đánh giá thẩm mỹ giao diện D. Kiểm thử thăm dò

Quiz 15 câu – Chương 5 (phần 2/3)

Câu 6. Trường hợp nào KHÔNG nên tự động hoá?
A. Smoke test B. Chức năng có giao diện đang thay đổi hằng ngày C. Test tính toán có kết quả xác định D. Test chạy với 200 bộ dữ liệu

Câu 7. Chi phí LỚN NHẤT và hay bị quên nhất của kiểm thử tự động là:
A. Mua bản quyền công cụ B. Bảo trì script khi ứng dụng thay đổi C. Tiền điện D. Đào tạo ban đầu

Câu 8. Test lúc Pass lúc Fail dù mã nguồn không đổi được gọi là:
A. Test hồi quy B. Test không ổn định (flaky test) C. Test khói D. Test tải

Câu 9. Cách xử lý ĐÚNG khi trang cần thời gian tải trước khi kiểm chứng:
A. Dùng lệnh ngủ cố định 5 giây B. Dùng chờ tường minh đợi phần tử xuất hiện C. Chạy lại script nhiều lần D. Bỏ qua bước kiểm chứng

Câu 10. Theo kim tự tháp kiểm thử, nên tự động hoá NHIỀU NHẤT ở tầng:
A. Giao diện B. API C. Đơn vị D. Chấp nhận

Quiz 15 câu – Chương 5 (phần 3/3)

Câu 11. Kiểm thử hướng dữ liệu (data-driven testing) nghĩa là:
A. Chỉ test cơ sở dữ liệu **B**. Tách dữ liệu ra ngoài, một script chạy với nhiều bộ dữ liệu C. Test bằng dữ liệu thật của khách hàng D. Tự sinh dữ liệu ngẫu nhiên

Câu 12. Nguyên tắc quan trọng để chạy test song song là các test phải:
A. Chạy đúng thứ tự **B**. Độc lập và tự chuẩn bị dữ liệu của mình C. Dùng chung một tài khoản D. Viết bằng cùng ngôn ngữ

Câu 13. Tiêu chí QUAN TRỌNG NHẤT khi chọn công cụ tự động hoá cho một nhóm là:
A. Công cụ nổi tiếng nhất **B**. Phù hợp loại ứng dụng và kỹ năng hiện có của nhóm C. Công cụ đắt nhất D. Công cụ mới ra gần đây

Câu 14. Bước nào trong vòng đời tự động hoá hay bị bỏ qua nhất, dẫn tới dự án thất bại?
A. Viết script B. Chọn công cụ C. Phân tích khả thi và bảo trì D. Chạy thử

Câu 15. Lợi ích nào của tự động hoá KHÔNG thể thay thế bằng test tay?
A. Tìm lỗi giao diện B. Mô phỏng 1.000 người dùng đồng. thời **C** Đánh giá trải nghiệm người dùng D. Kiểm thử thăm dò

Ngân hàng câu trắc nghiệm cho bài kiểm tra 45 phút – Chương 5

Câu 1. Một test case chạy tay mất 8 phút, viết script mất 90 phút, mỗi lần chạy script kể cả bảo trì mất 2 phút. Dự án còn 2 tháng, phát hành 1 lần/tuần. Kết luận ĐÚNG là:

A. Nên tự động hoá vì luôn có lợi B. Điểm hoà vốn 15 lần, dự án chỉ chạy khoảng 8 lần nên LỖ C. Điểm hoà vốn 8 lần nên có lãi D. Không đủ dữ liệu để kết luận

Câu 2. Sếp yêu cầu "tự động hoá 100% test case, bỏ hẳn test thủ công". Phản hồi CHUYÊN NGHIỆP NHẤT là:

A. Đồng ý và thực hiện B. Từ chối thẳng vì không khả thi **C.**Đề xuất tự động hoá theo mức ưu tiên kèm số liệu điểm hoà vốn và nêu phần không tự động hoá được D. Chuyển yêu cầu cho người khác

Câu 3. Một bộ test tự động sau 6 tháng có 30% test đỏ nhưng không ai sửa, cả nhóm đã bỏ không chạy nữa. Nguyên nhân GỐC RỄ là:

A. Công cụ kém **B.** Thiếu quy trình bảo trì và không chạy thường xuyên trong pipeline C. Tester không giỏi D. Ứng dụng quá phức tạp

Câu 4. Trong bộ 300 test case, nhóm chỉ tự động hoá được 50. Thứ tự ưu tiên HỢP LÝ NHẤT là:

A. Test khó nhất trước B. Test mới viết gần đây nhất trước **C**. Smoke test và hồi quy các chức năng lõi ổn định trước D. Test của chức năng đang phát triển trước

Phân tích hiệu quả và lập phương án tự động hoá

Tình huống. Website bán hàng của một công ty có bộ kiểm thử hồi quy gồm **200 test case**. Hiện chạy tay, mỗi test case trung bình **3 phút**. Công ty phát hành **2 lần mỗi tuần** và dự kiến duy trì hệ thống trong **2 năm**.

Ước tính khi tự động hoá: viết script mất **40 phút/test case**; mỗi lần chạy cả bộ tốn **25 phút** máy cộng **60 phút** người xem kết quả và bảo trì.

Yêu cầu.

a) Tính **chi phí chạy tay** và **chi phí tự động hoá** cho **1 năm đầu** (giả sử 50 tuần/năm). Kết luận nên hay không nên tự động hoá. (2,0 đ)

b) Nếu chỉ đủ nguồn lực tự động hoá **60 test case**, hãy nêu **tiêu chí chọn** và **ba nhóm test case** sẽ chọn theo thứ tự ưu tiên. (1,5 đ)

c) Nêu **hai rủi ro** chính của phương án tự động hoá này và **biện pháp giảm thiểu**. (1,5 đ)